

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN



CSC10014 - TƯ DUY TÍNH TOÁN
HKII: 2025 - 2026
HỆ THỐNG LƯU TRÚ
DU LỊCH THÔNG MINH

Lớp : CQ2024/6
Nhóm thực hiện : Nhóm 12
Giáo viên hướng dẫn : Hồ Tuấn Thanh
GVHDTH : Mai Anh Tuấn
Trợ giảng : Phạm Nguyễn Sơn Tùng

Thành Viên:

24120033: Đào Tiến Đạt.
24120040: Dương Hoài Đức.
24120304: Trần Thị Bích Hạnh.
24120310: Nguyễn Đình Hiếu.
24120483: Hoàng Văn Trường.

THÀNH PHỐ HỒ CHÍ MINH, NGÀY 20/6/2026

Lời Cảm Ơn

Trong quá trình thực hiện đồ án môn học, nhóm chúng em đã nhận được rất nhiều sự hỗ trợ và hướng dẫn quý báu từ các thầy cô và bạn bè.

Trước hết, chúng em xin gửi lời cảm ơn chân thành đến thầy **Hồ Tuấn Thanh** – giảng viên phụ trách học phần, đã cung cấp những kiến thức nền tảng cũng như tạo điều kiện để chúng em có cơ hội tiếp cận với các công nghệ và quy trình phát triển ứng dụng thực tế. Chúng em cũng xin chân thành cảm ơn thầy **Mai Anh Tuấn** – giảng viên hướng dẫn thực hành cùng các thầy cô trợ giảng đã tận tình hỗ trợ, giải đáp các thắc mắc trong quá trình học tập và thực hiện đồ án.

Thông qua học phần này, các thành viên trong nhóm không chỉ tích lũy được những kiến thức chuyên môn quý báu mà còn rèn luyện được tư duy giải quyết vấn đề, kỹ năng làm việc nhóm và khả năng vận dụng kiến thức vào các bài toán thực tế. Những kinh nghiệm thu được trong quá trình thực hiện đề tài sẽ là nền tảng quan trọng cho việc học tập và nghiên cứu trong tương lai.

Mặc dù đã nỗ lực hoàn thành đồ án với tinh thần học hỏi và trách nhiệm cao nhất, song do kiến thức và kinh nghiệm thực tiễn còn hạn chế nên bài làm khó tránh khỏi những thiếu sót. Chúng em rất mong nhận được những ý kiến đóng góp và nhận xét quý báu từ Thầy/Cô để có thể tiếp tục hoàn thiện kiến thức và kỹ năng của mình.

Cuối cùng, chúng em xin kính chúc quý Thầy/Cô luôn dồi dào sức khỏe, hạnh phúc và gặt hái được nhiều thành công trong sự nghiệp giảng dạy và nghiên cứu.

Trân trọng,
Các thành viên Nhóm 12

Table of Contents

1	Group Member	5
2	Idea	6
2.1	Problem Statement	6
2.2	Application Vision	6
2.3	Target Users	6
3	Problem Analysis	7
3.1	User Journey	7
3.2	Computational Problem Decomposition	7
3.3	Constraints, Assumptions, and Criteria	8
4	System Overview	9
4.1	Overall Architecture	9
4.2	Core Features	10
4.3	Feature Illustrations	10
4.4	Breakthrough Points	12
5	Pattern Recognition	13
5.1	Purpose and Approach	13
5.2	Application of Pattern Recognition Thinking	13
5.3	Summary	16
6	Abstraction	18
6.1	Role of Abstraction in System Architecture	18
6.2	Abstracted Components	18
6.3	Architectural Benefits and Balance Level	19
7	System / Algorithm Design	20
7.1	Architectural Design Goals	20
7.2	Overall Architecture	20
7.3	C4 Level 1: System Context	21
7.4	C4 Level 2: Container Architecture	21
7.5	Backend Modular Architecture	22
7.6	Frontend Architecture	22

7.7	Data Architecture	23
7.8	Key Runtime Workflows	23
7.9	Security and Access Control	24
7.10	Runtime Configuration and Environment Strategy	24
7.11	Architectural Decisions and Trade-offs	25
7.12	Current Limitations and Future Improvements	26
7.13	Summary	26
8	Implementation	26
8.1	From Design Model to Module Decomposition	26
8.2	Architecture Migration: Microservices → Modular Monolith	27
8.3	Challenges and Solutions	27
8.3.1	LLM Non-determinism	27
8.3.2	SSE Streaming via POST	28
8.3.3	Multi-Map Provider Integration	28
8.3.4	Vector Embedding and Prisma	28
8.3.5	Firebase Authentication	28
8.4	Other Challenges	28
8.5	Conclusion	29
9	Testing	29
9.1	Testing Strategy	29
9.2	Testing Scope	29
9.2.1	Frontend — UI and Interaction	29
9.2.2	Backend — Business Logic and Integration	30
9.2.3	Specific Scenarios and Mocking	30
9.3	Testing Results	31
9.4	Post-Testing Improvements	32
9.5	Future Development	32
10	Demo	33
11	Deployment	33
12	Logbook: Plan and Actual Workload	33
12.1	Plan Workload	33
12.2	Actual Workload	33
12.3	Workload Distribution	34

13 AI usage and declaration	34
14 Conclusion and Future Work	34
14.1 Conclusion	34
14.2 Acquired Knowledge and Skills	35
14.3 Future Development Directions	35

1. Group Member

STT	MSSV	Họ và Tên	Vai trò / Nhiệm vụ chính
1	24120033	Đào Tiến Đạt	
2	24120040	Dương Hoài Đức	
3	24120304	Trần Thị Bích Hạnh	
4	24120310	Nguyễn Đình Hiếu	
5	24120483	Hoàng Văn Trường	

2. Idea

2.1. Problem Statement

Hiện nay, việc tìm kiếm một nơi lưu trú phù hợp khi đi du lịch không hề đơn giản. Người dùng thường phải cân nhắc nhiều yếu tố như giá cả, vị trí, tiện nghi, chất lượng dịch vụ và các đánh giá từ những khách hàng trước đó. Quá trình này vẫn còn gặp nhiều khó khăn với các **điểm đau** cốt lõi:

- ❌ **Thông tin phân mảnh và Rời rạc:** Dữ liệu về khách sạn, homestay rải rác trên nhiều nền tảng, khiến người dùng mất nhiều thời gian tổng hợp và so sánh.
- ❌ **Bộ lọc cứng nhắc và Thiếu cá nhân hoá:** Các công cụ truyền thống chỉ cho phép lọc theo các tùy chọn có sẵn, không hỗ trợ tìm kiếm linh hoạt dựa trên nhu cầu tự nhiên và đặc thù của người dùng.
- ❌ **Độ tin cậy của dữ liệu thấp:** Đánh giá từ người dùng trước thường mang tính chủ quan, có thể bị thao túng bằng các đánh giá giả mạo và thiếu tính cập nhật thực tế tại thời điểm hiện tại.
- ❌ **Quá tải thông tin và Tốn thời gian:** Người dùng mất quá nhiều thời gian để đọc và đối chiếu thông tin từ nhiều nguồn trước khi có thể đưa ra quyết định chính xác.

💡 Giải pháp đề xuất

Nhóm xây dựng hệ thống tìm kiếm và gợi ý lưu trú thông minh, cho phép người dùng mô tả nhu cầu bằng **ngôn ngữ tự nhiên**. Hệ thống sẽ phân tích yêu cầu, tìm kiếm các địa điểm phù hợp, đánh giá, xếp hạng và đưa ra những gợi ý trực quan, giúp người dùng ra quyết định nhanh chóng.

2.2. Application Vision

Dự án không chỉ nhằm xây dựng một nền tảng thương mại thông thường, mà định hình lại cách người dùng tương tác với dữ liệu. Thông qua việc ứng dụng AI và kỹ thuật **RAG**, hệ thống kỳ vọng tạo ra một tiền đề cho tương lai, nơi AI đóng vai trò như một người điều phối đáng tin cậy.

2.3. Target Users

- 👤 **Khách du lịch tự túc:** Cần tìm kiếm nhanh chóng, so sánh giá để tối ưu hóa trải nghiệm.
- 👤 **Người dùng On-site:** Đang lưu trú tại địa điểm, sẵn sàng đóng góp thông tin thực tế và trả lời câu hỏi cộng đồng.
- 👤 **Người ưu tiên cá nhân hóa:** Thích tương tác với AI thay vì tốn hàng giờ với các bộ lọc phức tạp.

3. Problem Analysis

3.1. User Journey

Thay vì tiếp cận theo hướng kỹ thuật khô khan, nhóm thiết kế hệ thống thông qua chuỗi câu hỏi dẫn dắt từ góc nhìn người dùng:

- ❓ Người dùng sẽ làm gì với ứng dụng của tôi?**
→ Người dùng cần tìm địa điểm lưu trú.
- ❓ Người dùng có thể tìm kiếm địa điểm lưu trú như thế nào?**
→ Thiết kế giao diện tìm kiếm hoặc phương thức tìm kiếm linh hoạt.
- ❓ Vậy làm sao để phương thức tìm kiếm có thể cho ra kết quả tốt?**
→ Chúng ta sẽ đánh giá trên các tiêu chí, như vậy chúng ta cần trích xuất ý định của người dùng.
- ❓ Vậy việc trích xuất nên được thực hiện thế nào?**
→ Có thể cho phép người dùng chọn hoặc thông qua tác tử AI được cài đặt.
- ❓ Nếu người dùng sau khi đã có các địa điểm phù hợp với nhu cầu của họ, làm cách nào để hệ thống có thể giúp người dùng ra quyết định cuối cùng như thế nào?**
→ Có thể cài đặt tính năng giúp người dùng so sánh các địa điểm, hỗ trợ ra quyết định cuối.
- ❓ Vậy nếu người dùng muốn tìm hiểu sâu hơn về một địa điểm thì sao?**
→ Cài đặt tính năng AI đưa ra phân tích chi tiết về một địa điểm cụ thể, cho phép người dùng lựa chọn các tiêu chí để phân tích.
- ❓ Việc sử dụng LLM có thể không đạt được độ chính xác cao, liệu nguồn thông tin nào uy tín hơn không?**
→ Sử dụng nguồn thông tin từ chính người dùng, có thể áp dụng chính sách 'người dùng onsite' để tăng độ tin cậy.
- ❓ Vậy với các tính năng như trên, có thể tối giản giao diện để người dùng có thể tương tác nhanh chóng và dễ dàng, phù hợp với nhiều người dùng ở các độ tuổi, trình độ khác nhau hay không?**
→ Sử dụng LLM đóng vai trò điều phối trung gian, người dùng sẽ tương tác trực tiếp với LLM, LLM sẽ trích xuất ý định người dùng và tiến hành các yêu cầu một cách tự động.

3.2. Computational Problem Decomposition

Từ nhu cầu thực tế, hệ thống được phân rã thành các module kỹ thuật chuyên biệt:

- ⚙️ Intent Parsing:** LLM chuyển đổi ngôn ngữ tự nhiên thành JSON cấu trúc.

- ⚙️ **Data Aggregation:** Truy xuất đồng thời DB nội bộ và API đối tác.
- ⚙️ **Semantic Search:** Số hóa văn bản thành vector nhúng (với pgvector).
- ⚙️ **Workflow Engine:** Xây dựng khung làm việc cứng kiểm soát AI.

3.3. Constraints, Assumptions, and Criteria

Để đảm bảo tính khả thi trong phạm vi đề án, hệ thống được thiết kế và đánh giá dựa trên các khía cạnh rõ ràng sau:

🔗 1. Ràng buộc

- ▶ **Về mặt kiến trúc:** Lựa chọn mô hình *Modular Monolith* để giảm thiểu chi phí quản lý và vận hành trong giai đoạn MVP, đồng thời vẫn giữ được tính phân tách cao giúp dễ dàng mở rộng sang Microservices sau này.
- ▶ **Về mặt tài nguyên:** Hệ thống chuyển giao toàn bộ tải tính toán AI nặng nề sang các API của bên thứ ba (như Groq). Nhờ đó, hệ thống lõi có thể chạy cục bộ mượt mà trên phần cứng cơ bản không yêu cầu GPU.
- ▶ **Về phạm vi hỗ trợ:** Ứng dụng ở giai đoạn này tập trung vào thị trường Việt Nam. Hệ thống tập trung xác định các địa điểm lưu trú trong nước, yêu cầu AI phải hỗ trợ tiếng Việt tốt và hiểu rõ ngữ cảnh văn hóa địa phương.

💡 2. Giả định

- ▶ Khả năng lập luận, phân tích ngữ nghĩa và đưa ra quyết định của Mô hình ngôn ngữ lớn là yếu tố khách quan, tính chính xác phụ thuộc vào API cung cấp chứ không bị giới hạn bởi hệ thống nội bộ.
- ▶ Dữ liệu thu thập từ các nhà cung cấp bên thứ ba (Goong, SerpAPI) luôn đảm bảo độ tin cậy về mặt nội dung và ổn định về thời gian phản hồi (uptime).

☰ 3. Tiêu chí đánh giá

- ▶ **Sự hài lòng của người dùng:** Đánh giá mức độ thoải mái, mức độ cá nhân hóa và sự hài lòng tổng thể khi người dùng trải nghiệm các tính năng tìm kiếm, trò chuyện cùng AI trên nền tảng.
- ▶ **Tính hữu dụng:** Ứng dụng phải giải quyết được triệt để bài toán tìm kiếm chỗ ở. Giao diện cần trực quan, dễ thao tác và có luồng tương tác mượt mà phù hợp với nhiều nhóm đối tượng.
- ▶ **Độ chính xác của thông tin:** Đảm bảo chất lượng dữ liệu trả về. Danh sách địa điểm phải sát thực tế, và đặc biệt câu trả lời phân tích chi tiết từ AI phải chuẩn xác, tuyệt đối không được xảy ra tình trạng ảo giác.

4. System Overview

Hệ thống là nền tảng tìm kiếm và quản lý lưu trữ thông minh, kết hợp tra cứu truyền thống và sức mạnh Trí tuệ Nhân tạo. Dự án mang đến trải nghiệm đột phá qua giao diện hội thoại và tìm kiếm ngữ nghĩa.

4.1. Overall Architecture

Hệ thống được thiết kế theo mô hình **Modular Monolith** kết hợp với cơ chế điều phối tác tử AI.

❓ Tại sao chọn Modular Monolith?

Lý do nhóm ưu tiên sử dụng **Modular Monolith** là nhằm giảm thiểu tối đa sự phức tạp trong quá trình triển khai và vận hành hệ thống ở giai đoạn nguyên mẫu. Với kiến trúc này, toàn bộ hệ thống được đóng gói chung để dễ quản lý, nhưng bên trong lại được phân tách nghiêm ngặt thành các phân hệ nghiệp vụ độc lập. Cách thiết kế này giúp mã nguồn luôn rõ ràng, ngăn chặn sự chồng chéo logic, tiết kiệm tài nguyên hệ thống, và đặc biệt là tạo tiền đề thuận lợi để dễ dàng bóc tách thành các dịch vụ độc lập (Microservices) trong tương lai khi dự án thực sự cần mở rộng quy mô.

Về tổng thể, kiến trúc bao gồm ba tầng chính:

- 🏠 **Tầng Giao diện:** Tập trung tối ưu hóa trải nghiệm người dùng, cung cấp bản đồ tương tác, không gian hỏi đáp cộng đồng và đặc biệt là khung trò chuyện AI với khả năng nhận luồng phản hồi liên tục theo thời gian thực.
- 🏢 **Tầng Xử lý (Nghiệp vụ và AI):** Đóng vai trò là trung tâm điều phối, bao gồm các phân hệ quản lý người dùng, địa điểm và đánh giá. Điểm nhấn là phân hệ AI hoạt động như một cỗ máy điều phối quy trình công việc, chịu trách nhiệm phân tích ý định từ ngôn ngữ tự nhiên và tự động kích hoạt các công cụ tương ứng.
- 📦 **Tầng Dữ liệu:** Chịu trách nhiệm lưu trữ an toàn toàn bộ dữ liệu quan hệ của nền tảng. Đồng thời, tầng này cũng tích hợp khả năng lưu trữ và truy vấn các vector nhúng, làm cốt lõi cho tính năng tìm kiếm ngữ nghĩa và truy xuất dữ liệu tăng cường.

4.2. Core Features

STT	Tính Năng	Mô Tả Ngắn
1	 Tìm kiếm và Gợi ý	Trải nghiệm tìm kiếm đa chiều kết hợp linh hoạt giữa bộ lọc và hệ thống gợi ý cá nhân hóa.
2	 Trợ lý Ảo AI	Chatbot thông minh hỗ trợ tìm kiếm và chất lọc thông tin lưu trữ qua hội thoại tự nhiên.
3	 Quản lý Cộng đồng	Không gian tương tác, hỏi đáp và chia sẻ nhận xét trực tiếp tại trang của từng địa điểm.
4	 Xác nhận On-site	Tính năng check-in độc đáo để xác nhận người dùng đang có mặt tại địa điểm thực tế.
5	 Đóng góp Nội dung	Trao quyền cho người dùng tải lên hình ảnh và cập nhật tình trạng mới nhất của chỗ ở.
6	 Lưu trữ Địa điểm	Công cụ cá nhân để đánh dấu, lưu lại và so sánh các danh sách địa điểm yêu thích.

4.3. Feature Illustrations

Thay vì tiếp cận theo hướng kỹ thuật, hệ thống được thiết kế dựa trên quá trình tối ưu hóa trải nghiệm thực tế của người dùng thông qua các tính năng sau:

 1. Trải nghiệm Tìm kiếm Linh hoạt

Hệ thống khắc phục hạn chế của các bộ lọc tìm kiếm truyền thống bằng cách cho phép người dùng nhập trực tiếp các yêu cầu phức tạp dưới dạng ngôn ngữ tự nhiên. Ví dụ: *"Tìm một home-stay yên tĩnh ở Đà Lạt cho gia đình 4 người, có bếp nướng BBQ và giá dưới 2 triệu."* Thông qua khả năng phân tích ngữ nghĩa, hệ thống sẽ trích xuất chính xác các tiêu chí và trả về danh sách địa điểm phù hợp nhất.



 2. Hỏi đáp Trực tiếp cùng Trợ lý Ảo

Illustration (0.6 width)
(Chat AI)

Nhằm giảm thiểu thời gian đọc và tổng hợp thông tin từ hàng loạt đánh giá dài, người dùng có thể đặt câu hỏi trực tiếp cho AI tại trang thông tin của điểm lưu trú. Ví dụ: "*Khách sạn này ăn sáng có đa dạng không? Tốc độ mạng wifi có ổn định không?*". Trợ lý ảo sẽ tự động phân tích các đánh giá thực tế từ cơ sở dữ liệu để đưa ra câu trả lời khách quan, ngắn gọn và bám sát thắc mắc của người dùng.

✔ 3. Chống Thông tin Sai lệch qua cơ chế “On-site”

Để giải quyết rủi ro sai lệch thông tin giữa quảng cáo và thực tế, hệ thống tích hợp tính năng xác thực "Check-in" dành riêng cho những khách hàng đang có mặt tại điểm lưu trú. Những hình ảnh, đánh giá hoặc câu trả lời được cung cấp bởi nhóm người dùng "On-site" này sẽ được gắn nhãn xác thực và ưu tiên hiển thị, qua đó nâng cao độ tin cậy của thông tin trên nền tảng.

Illustration (0.6 width)
(On-site)

👥 4. Tương tác Cộng đồng Trực tiếp

Illustration (0.6 width)
(Community)

Mỗi địa điểm trên nền tảng được tổ chức như một diễn đàn độc lập. Người dùng có thể đặt các câu hỏi đặc thù và nhận phản hồi trực tiếp từ những du khách đã hoặc đang lưu trú tại đó. Tính năng này tạo ra một vòng lặp chia sẻ thông tin liên tục, giúp dữ liệu luôn được cập nhật và phản ánh đúng tình trạng hiện tại của điểm đến.

4.4. Breakthrough Points

Các điểm đổi mới nổi bật

- ✓ **AI-Driven Workflow:** AI không chỉ sinh chữ. Nó là "bộ định tuyến" gọi API, tính toán logic và vẽ thẳng kết quả lên UI.
- ✓ **Semantic Search và Vector DB:** Khám phá chỗ ở thông qua độ tương đồng Cosine của ngữ nghĩa, vượt ra khỏi giới hạn so khớp từ khóa cứng nhắc.
- ✓ **On-site Insights:** Giải quyết triệt để sự sai lệch giữa quảng cáo và thực tế bằng cách ưu tiên tương tác từ người dùng thực chứng.

5. Pattern Recognition

5.1. Purpose and Approach

Trong dự án này, Nhận diện mẫu đóng vai trò thiết yếu để định hình kiến trúc hệ thống. Thay vì giải quyết từng vấn đề một cách đơn lẻ, nhóm đã quan sát các trường hợp lặp lại trong quá trình vận hành, so sánh chúng, gom nhóm những điểm tương đồng và khái quát hóa thành các khuôn mẫu thiết kế hữu ích. Các phương pháp tổng quát như đối chiếu nhiều trường hợp, phân tích sự lặp lại và kiểm chứng bằng các kịch bản kiểm thử (test cases) đã được áp dụng triệt để.

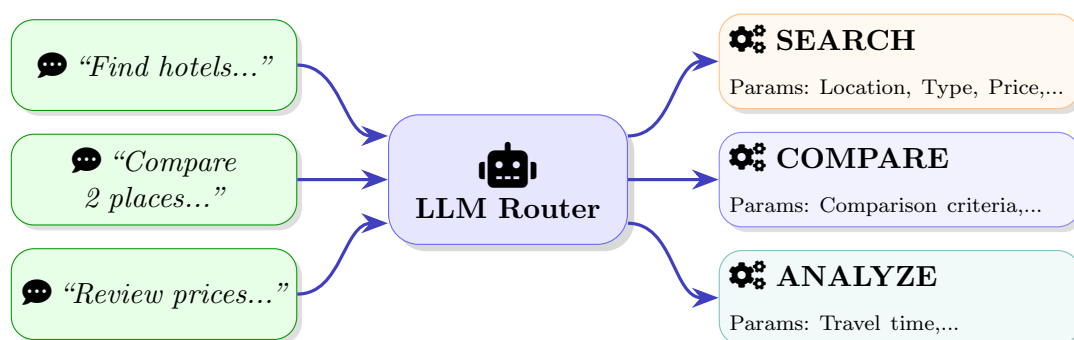
Việc lựa chọn cách tiếp cận này xuất phát từ đặc thù của hệ thống: phải tiếp nhận vô số câu lệnh tự nhiên đa dạng, vận hành nhiều luồng xử lý song song, tổng hợp thông tin từ nhiều nguồn và đối mặt với hàng loạt tình huống lỗi ngoại vi. Nếu xử lý từng trường hợp riêng lẻ, hệ thống sẽ trở nên rời rạc, thiếu đồng bộ và rất khó mở rộng. Nhờ đúc kết được 6 quy luật cốt lõi dưới đây, nhóm đã có cơ sở vững chắc để đưa ra các quyết định thiết kế kiến trúc chuẩn xác.

5.2. Application of Pattern Recognition Thinking

Q 1. Nhận diện mục đích tìm kiếm

Thực tế cho thấy, dù người dùng có cách diễn đạt đa dạng khi tìm kiếm nơi lưu trú, các câu lệnh của họ luôn xoay quanh những thông tin cốt lõi lặp lại như vị trí, ngân sách, loại chỗ ở, tiện nghi và các yêu cầu mềm khác.

Quy tắc tổng quát được rút ra là: mọi yêu cầu phức tạp đều có thể được bóc tách thành các thông tin cụ thể kết hợp với mục đích cốt lõi. Quy tắc này giúp nhóm thiết kế một hệ thống trí tuệ nhân tạo có khả năng tự động đọc hiểu câu văn tự nhiên của người dùng, từ đó phân tích và chuyển đổi thành những điều kiện tìm kiếm chuẩn xác để hệ thống có thể xử lý dễ dàng.

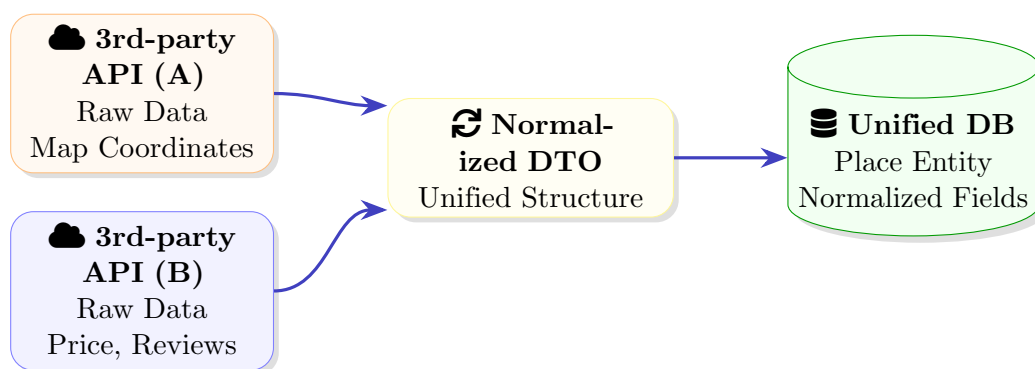


Hình 1: **Query Pattern** — Chuẩn hóa đa dạng câu hỏi tự nhiên thành các tham số định tuyến.

2. Nhận diện cấu trúc thông tin lưu trú

Trong quá trình thu thập thông tin từ nhiều nguồn khác nhau, nhóm phát hiện ra một sự tương đồng lớn về mặt nội dung. Dù định dạng dữ liệu ban đầu từ các bên cung cấp có sự khác biệt, mọi điểm lưu trữ đều chia sẻ những thông tin nền tảng giống nhau.

Dựa trên quy tắc này, nhóm quyết định thiết kế một cơ chế trung gian có nhiệm vụ thu gom, sắp xếp và đồng nhất mọi định dạng dữ liệu trước khi lưu trữ chính thức vào hệ thống cốt lõi.

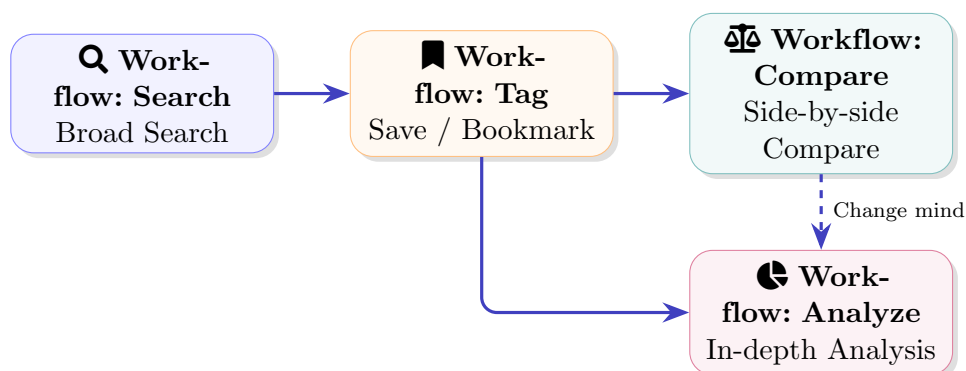


Hình 2: **Data Pattern** — Nhận diện mẫu thuộc tính từ các nguồn phân mảnh để ánh xạ về mô hình chuẩn hóa Place.

🔗 3. Nhận diện quy trình ra quyết định

Hành vi của người dùng trong quá trình lựa chọn chỗ ở hiếm khi diễn ra theo một đường thẳng. Thực tế cho thấy một thói quen lặp lại: Người dùng sẽ bắt đầu bằng việc tìm kiếm diện rộng, sau đó đánh dấu những nơi ưng ý, và cuối cùng là so sánh hoặc đánh giá chi tiết.

Từ đặc điểm này, nhóm đã xây dựng sẵn các quy trình xử lý cốt lõi được liên kết chặt chẽ với nhau nhằm hỗ trợ trọn vẹn toàn bộ quá trình cân nhắc và ra quyết định phức tạp của người dùng.



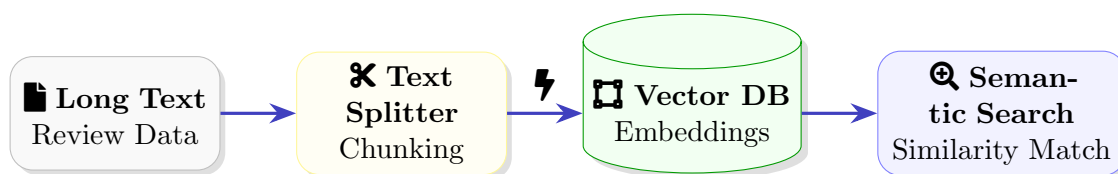
Hình 3: **Decision-making Pattern** — Khớp nối Workflow hệ thống với mẫu hành vi ra quyết định lặp lại.

★ 4. Nhận diện phương pháp xử lý lưu trữ thông tin

Trong thực tiễn, các luồng thông tin dạng ngôn ngữ tự nhiên như đánh giá, nhận xét hay mô tả của người dùng có mức độ phân hóa vô cùng đa dạng. Mặc dù không theo một cấu trúc cố định, những thông tin này lại đóng vai trò tối quan trọng: chúng thường xuyên được sử dụng làm ngữ cảnh hỗ trợ, cung cấp dữ liệu chi tiết cho Mô hình ngôn ngữ lớn nhằm tối ưu hóa và gia tăng độ chính xác cho câu trả lời.

Nhận thức được điều này, nhóm đánh giá việc thiết lập một khuôn mẫu chung để xử lý thống nhất các luồng ngôn ngữ tự do là một yêu cầu cấp thiết. Từ đó đã dẫn đến **ý tưởng then chốt**: tiến hành "số hóa" toàn bộ thông tin dạng ngôn ngữ tự nhiên.

Việc chuyển đổi câu chữ thành các dải số học không chỉ giúp máy tính hiểu được ý nghĩa sâu xa mà còn cho phép trích xuất chính xác các ngữ cảnh cần thiết nhất để phục vụ cho hệ thống xử lý.



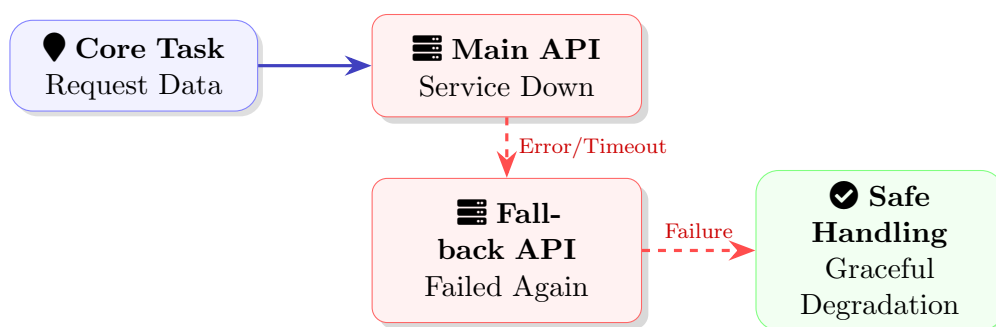
Hình 4: **RAG Pattern** — Xử lý văn bản tự do thành các khối thông tin (chunks) truy xuất được bằng vector ngữ nghĩa.

⚠ 5. Nhận diện rủi ro hệ thống

Việc kết nối với các dịch vụ bên ngoài luôn tiềm ẩn những rủi ro ngất quãng như quá tải hoặc mất kết nối. Điều này dẫn đến **quy tắc thiết kế**: sự cố từ bên ngoài là không thể tránh khỏi, do đó hệ thống cốt lõi bắt buộc phải có khả năng tự bảo vệ và phục hồi.

Khi một dịch vụ chính gặp sự cố, hệ thống sẽ tự động kích hoạt cơ chế chuyển đổi sang dịch vụ dự phòng.

Trong trường hợp mọi phương án đều thất bại, hệ thống sẽ chủ động xử lý lỗi một cách khéo léo và trả về kết quả an toàn nhằm đảm bảo trải nghiệm của người dùng không bị gián đoạn.

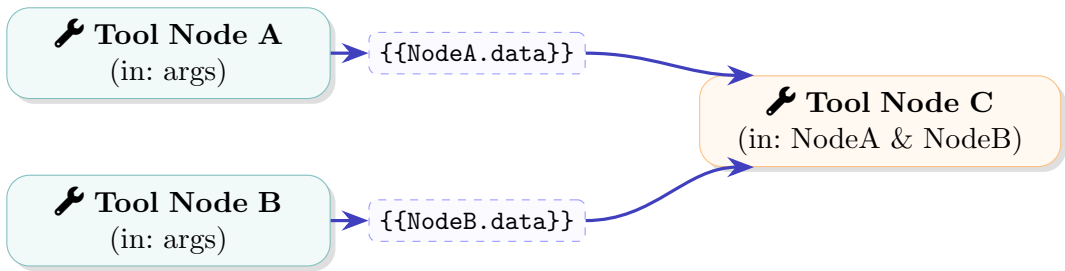


Hình 5: **Failure Pattern** — Cơ chế Fallback đối phó với mẫu lỗi lặp lại từ các nhà cung cấp API ngoại vi.

🔗 6. Nhận diện luồng liên kết workflow

Các quy trình xử lý phức tạp thực chất là một chuỗi các bước thực hiện, nơi kết quả của bước trước trở thành nguyên liệu đầu vào cho bước sau. Nhóm đã nhận diện được đặc điểm này và giải quyết bằng cách thiết lập một cơ chế truyền dữ liệu linh hoạt làm cầu nối giao tiếp giữa các tác vụ.

Quyết định thiết kế này giúp tạo ra một luồng công việc linh động, cho phép hệ thống dễ dàng lắp ráp, đảo vị trí hoặc thêm mới các thao tác hỗ trợ mà không làm phá vỡ cấu trúc tổng thể.



Hình 6: **Workflow Tool Pattern** — Nhận diện mẫu phụ thuộc dữ liệu bằng cách sử dụng String Template giữa các Tool.

5.3. Summary

Bảng 1: **Pattern Recognition Summary**

📖 Mẫu được nhận diện	💡 Quy tắc tổng quát	⚙️ Quyết định thiết kế
Query Pattern Cách diễn đạt đa dạng nhưng luôn xoay quanh các tiêu chí, ý định cốt lõi	Đa phần yêu cầu phức tạp đều có thể được phân tách thành mục đích chính và các điều kiện cụ thể.	Sử dụng LLM để đọc hiểu và chuyển đổi câu văn tự nhiên thành các tiêu chí tìm kiếm chuẩn xác.
Data Pattern Thông tin từ nhiều nguồn luôn có sự tương đồng về thuộc tính nền tảng.	Mọi thông tin thô dù khác biệt định dạng đều có thể được ánh xạ về một cấu trúc chung.	Xây dựng cơ chế trung gian để thu gom, làm sạch và đồng nhất toàn bộ dữ liệu trước khi lưu trữ.
Decision-making Pattern Quá trình chọn lọc thường đi qua nhiều bước cân nhắc, đối chiếu.	Hành vi ra quyết định hiếm khi đi đường thẳng mà là một chuỗi hành động lặp lại.	Cung cấp sẵn các luồng xử lý liên kết chặt chẽ (lưu trữ, so sánh, phân tích) để hỗ trợ quá trình cân nhắc.

Bảng 1: **Pattern Recognition Summary** (tiếp tục)

 Mẫu được nhận diện	 Quy tắc tổng quát	 Quyết định thiết kế
RAG Pattern Cơ chế xử lý đối với dữ liệu dạng ngôn ngữ tự nhiên	Các đoạn văn bản dài cần được phân tích theo ngữ nghĩa để dễ dàng trích xuất thông tin trọng tâm.	Số hóa văn bản thành các dải số học và sử dụng kỹ thuật tìm kiếm tương đồng để tự động giải đáp.
Failure Pattern Các dịch vụ bên ngoài thường xuyên gặp sự cố mất kết nối hoặc quá tải	Rủi ro từ ngoại vi là không thể tránh khỏi, hệ thống bắt buộc phải biết tự bảo vệ.	Thiết lập cơ chế tự động chuyển sang dịch vụ dự phòng hoặc xử lý lỗi khéo léo để không gây gián đoạn.
Workflow Tool Pattern Các quy trình phức tạp luôn vận hành theo chuỗi liên kết thông tin	Các tools được sử dụng có thể được nối với nhau để tạo ra một workflow hoàn chỉnh	Tạo ra cơ chế truyền dữ liệu linh hoạt, cho phép dễ dàng tháo lắp và sắp xếp các bước xử lý một cách trơn tru.

Tổng kết lại, các khuôn mẫu được nhận diện đã giúp nhóm biến những hiện tượng lặp lại trong thực tế thành các quy tắc thiết kế chuẩn mực. Hệ thống hoàn toàn không được thiết kế như một tập hợp các tính năng rời rạc, mà được tổ chức một cách có hệ thống xoay quanh chính các cấu trúc lặp lại của bài toán gốc. Việc áp dụng thành công tư duy nhận diện mẫu mang lại tác dụng to lớn: giúp toàn bộ kiến trúc trở nên nhất quán hơn, dễ dàng nâng cấp và mở rộng, tạo thuận lợi tối đa cho công tác kiểm thử, và quan trọng nhất là đáp ứng chính xác nhu cầu thực tiễn của người dùng.

6. Abstraction

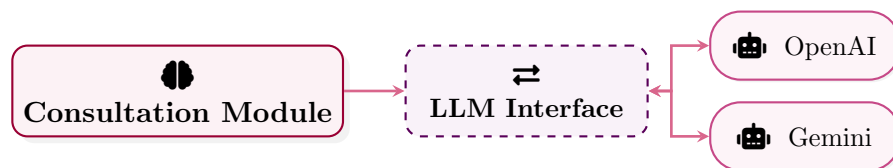
6.1. Role of Abstraction in System Architecture

Đây là một nền tảng lưu trữ du lịch thông minh, đòi hỏi sự phối hợp liên tục giữa nhiều thành phần có độ phức tạp cao như mô hình ngôn ngữ lớn (LLM), dịch vụ bản đồ, giao tiếp dữ liệu địa điểm, cơ sở dữ liệu vector và các cơ chế xác thực. Nếu để hệ thống phụ thuộc trực tiếp vào đặc tả chi tiết của từng nhà cung cấp dịch vụ, toàn bộ kiến trúc sẽ trở nên cứng nhắc, khó mở rộng và dễ đứt gãy mỗi khi có sự thay đổi từ bên ngoài. Để giải quyết rào cản này, nhóm phát triển đã áp dụng mạnh mẽ tư duy trừu tượng hóa (Abstraction) nhằm thiết lập các lớp khái niệm ổn định, che giấu hoàn toàn các chi tiết triển khai cụ thể phía sau. Tư duy này cho phép hệ thống giao tiếp với các dịch vụ cốt lõi dựa trên chức năng thiết yếu của chúng thay vì bị trói buộc vào một công nghệ hay nhà cung cấp nhất định.

6.2. Abstracted Components

Trong quá trình xây dựng kiến trúc hệ thống, tư duy trừu tượng hóa được áp dụng xuyên suốt trên nhiều khía cạnh khác nhau:

1. Năng lực xử lý ngôn ngữ (LLM): Hệ thống không bị ràng buộc vào một nhà cung cấp AI cụ thể nào. Ở mức kiến trúc, LLM đơn thuần được xem như một tác nhân cung cấp khả năng thấu hiểu ngôn ngữ tự nhiên, phân tích ý định người dùng và tự động tạo ra phản hồi. Mọi sự khác biệt phức tạp về mô hình, định dạng giao tiếp API, tốc độ phản hồi hay cơ chế truyền dữ liệu liên tục (streaming) đều được che giấu kỹ lưỡng. Nhờ vậy, luồng xử lý chính luôn được giữ ở trạng thái ổn định ngay cả khi tiến hành chuyển đổi nhà cung cấp hay nâng cấp mô hình.



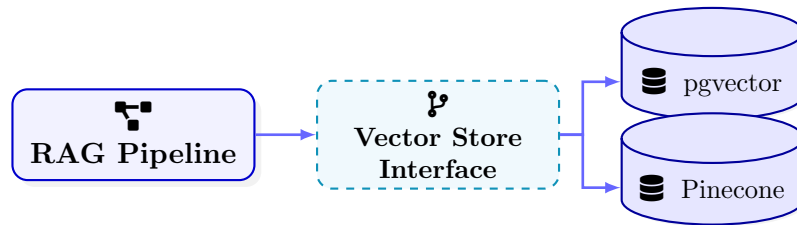
Hình 7: Trừu tượng hóa Mô hình Ngôn ngữ (LLM)

2. Dịch vụ bản đồ và dữ liệu địa điểm: Nhóm nhận thức rõ sự khác biệt tiềm tàng về cấu trúc dữ liệu, phương thức xác thực và độ tin cậy từ các nhà cung cấp khác nhau. Giải pháp được đưa ra là trừu tượng hóa mạng lưới này thành các *nguồn dữ liệu địa điểm tiêu chuẩn*. Nhờ đó, các phân hệ cốt lõi chỉ cần làm việc với một tập dữ liệu đã được tinh gọn và chuẩn hóa thống nhất, loại bỏ sự cần thiết phải thấu hiểu logic nội bộ phức tạp của từng API riêng biệt.



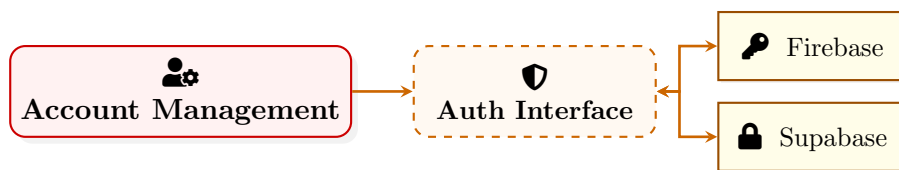
Hình 8: Trừu tượng hóa Dữ liệu Bản đồ

3. Trích xuất và truy vấn thông tin (RAG): Kiến trúc tổng thể không phụ thuộc chặt chẽ vào cách thức vận hành bên trong của một công nghệ vector cụ thể. Thay vào đó, toàn bộ quy trình RAG được thiết kế thành một thành phần độc lập mang trọng trách truy xuất ngữ cảnh liên quan. Những chi tiết kỹ thuật chuyên sâu về mô hình biểu diễn (embedding model), phương thức lưu trữ vector hay cơ chế tính toán độ tương đồng đều được ẩn giấu. Điều tối quan trọng là việc nhận lại đúng ngữ cảnh cần thiết để tổng hợp thành tư vấn đáng tin cậy.



Hình 9: Trừu tượng hóa Vector Store

4. Hạ tầng quản lý danh tính và lưu trữ: Các thành phần Authentication và Storage cũng tuân thủ nguyên tắc trừu tượng hóa. Dù đây là những dịch vụ dễ biến động tùy theo môi trường triển khai, chúng vẫn được chuyển đổi thành các khái niệm cung cấp tài nguyên thuần túy. Kết quả là, mọi mô-đun nghiệp vụ đều vận hành trơn tru mà không cần trực tiếp thao tác với cấu trúc danh tính, SDK hay giao thức độc quyền của từng nền tảng.



Hình 10: Trừu tượng hóa Quản lý Danh tính

6.3. Architectural Benefits and Balance Level

Gia tăng sự linh hoạt: Việc vận dụng triệt để Abstraction mang lại sự suy giảm đáng kể mức độ phụ thuộc chéo giữa các phân hệ, tạo không gian linh hoạt tối đa để thay thế hoặc luân chuyển nhà cung cấp. Sự phân tách này giúp hệ thống trở nên dễ kiểm thử, dễ bảo trì, và mở ra tiềm năng mở rộng không giới hạn trong tương lai. Hệ quả tất yếu là rủi ro kỹ thuật giảm thiểu tối đa, đặc biệt là khi các API bên ngoài thay đổi.

Cân bằng mức độ trừu tượng: Dù mang lại giá trị to lớn, nhóm phát triển ý thức được rằng trừu tượng hóa không phải là công cụ lạm dụng bừa bãi. Việc trừu tượng hóa quá

mức sẽ biến hệ thống thành một cấu trúc mơ hồ, trong khi quá ít lại khiến logic nghiệp vụ bị gắn chặt với các nền tảng. Nhóm đã thận trọng lựa chọn điểm cân bằng: *mọi lớp trừu tượng được tạo ra đều phải gắn liền với một định hướng kiến trúc rõ ràng*. Mỗi lớp trung gian chỉ tồn tại nếu nó thực sự giải quyết bài toán thay thế nhà cung cấp, chuẩn hóa dữ liệu, hoặc gia tăng khả năng mở rộng.

7. System / Algorithm Design

7.1. Architectural Design Goals

Hệ thống được thiết kế nhằm mang lại trải nghiệm khám phá các địa điểm lưu trú và ẩm thực trực quan thông qua bản đồ. Ngoài việc cung cấp thông tin, nền tảng tập trung mạnh vào sự tương tác của những người dùng đã xác thực thông qua các tính năng đánh giá, hỏi đáp, đánh dấu yêu thích và đặc biệt là cơ chế xác thực vị trí vật lý. Nền tảng kết nối hài hòa giữa dữ liệu địa điểm từ các nguồn bên ngoài và dữ liệu cộng đồng được lưu trữ bền vững bên trong. Trợ lý trí tuệ nhân tạo được tích hợp sâu vào hệ thống nhằm cung cấp các tư vấn chuẩn xác mà không làm gián đoạn hoặc cản trở thao tác của người dùng. Xét trên phương diện vận hành, hệ thống phải đảm bảo khả năng triển khai và bảo trì dễ dàng, phù hợp với năng lực quản lý của một nhóm phát triển sinh viên.

Để đạt được những mục tiêu trên, nhóm quyết định áp dụng kiến trúc Modular Monolith thay vì kiến trúc Microservices. Quyết định này giúp tối giản hóa đáng kể quy trình triển khai và môi trường local development, đồng thời giữ được một ranh giới database transaction chung duy nhất. Nhờ vậy, độ phức tạp trong vận hành được giảm tải tối đa, nhưng hệ thống vẫn đảm bảo được sự tách biệt ranh mạch giữa các lĩnh vực nghiệp vụ thông qua cấu trúc logic module ở backend.

7.2. Overall Architecture

Hệ thống được phân tầng rõ rệt thành các lớp chuyên biệt, cấu thành một khối kiến trúc Modular Monolith hoàn chỉnh.

Frontend: Sử dụng bộ công cụ React kết hợp Vite và Tailwind CSS. Lớp này đảm nhận toàn bộ giao diện tương tác, bao gồm bản đồ tìm kiếm, thẻ thông tin địa điểm, trang chi tiết địa điểm với khu vực hỏi đáp và đánh giá. Nó cũng quản lý hiển thị trạng thái có mặt vật lý, chức năng lưu địa điểm và trang hồ sơ cá nhân.

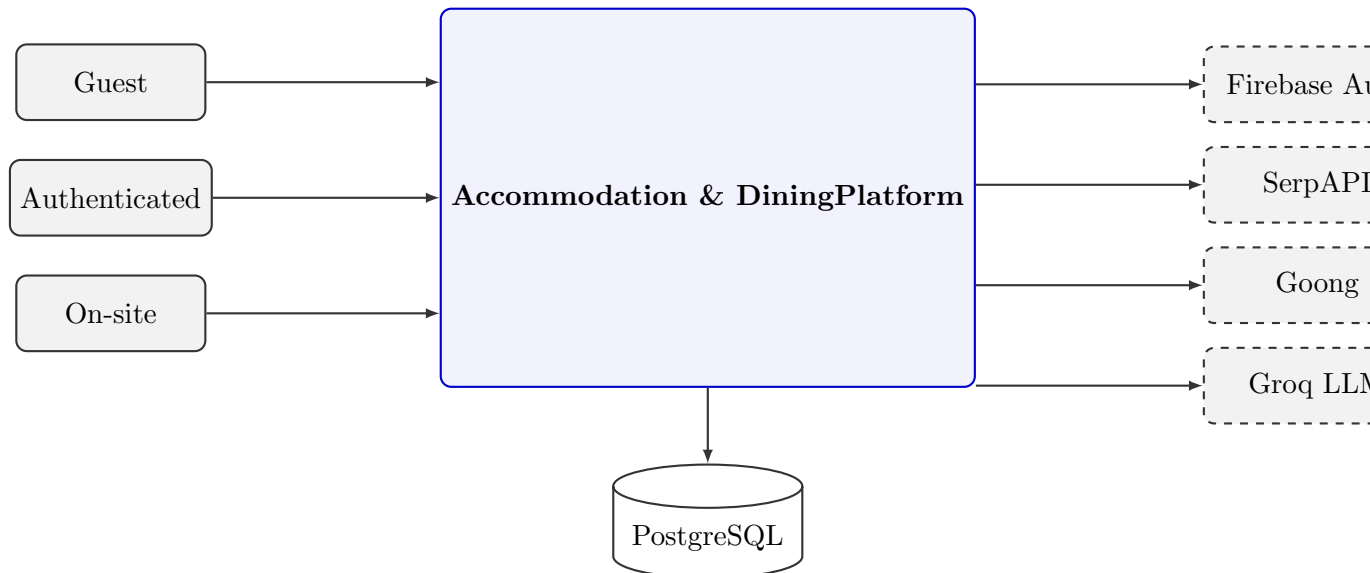
Backend/Application Layer: Được xây dựng trên nền tảng NestJS. Lớp này đóng vai trò trung tâm điều phối mọi luồng nghiệp vụ. Nó xác thực danh tính người dùng, điều hướng các luồng tìm kiếm, đồng bộ hóa dữ liệu địa điểm bên ngoài vào cơ sở dữ liệu nội bộ. Ngoài ra, lớp này quản lý toàn bộ vòng đời của các bài đánh giá, chuỗi hỏi đáp, điều phối các phản hồi từ trí tuệ nhân tạo, xác minh vị trí vật lý, và lưu trữ danh sách yêu thích.

Data Layer: Sử dụng hệ quản trị PostgreSQL kết hợp Prisma ORM. Đây là nơi lưu trữ bền vững mọi thông tin về người dùng, địa điểm, đánh giá, câu hỏi, câu trả lời, bản ghi trạng thái có mặt, danh sách yêu thích và cả lịch sử hội thoại của trợ lý thông minh.

External Integrations: Hệ thống tận dụng các dịch vụ chuyên biệt từ bên ngoài nhằm tối ưu hiệu năng: Firebase phục vụ xác thực danh tính; SerpAPI cung cấp dữ liệu tìm kiếm địa điểm; Goong đảm nhận định vị tọa độ; và mô hình LLM Groq xử lý suy luận cho trợ lý thông minh. Các lớp này tương tác chặt chẽ thông qua các giao thức mạng bảo mật, trong đó lớp Ứng dụng đóng vai trò làm lá chắn trung gian.

7.3. C4 Level 1: System Context

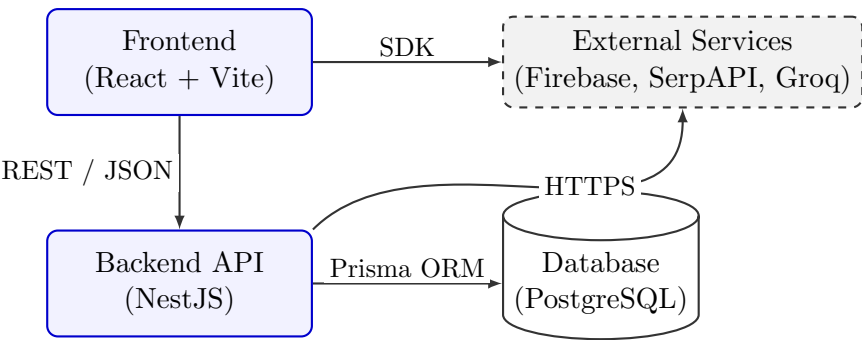
Hệ thống phục vụ ba nhóm đối tượng chính: Khách vắng lai, Người dùng đã xác thực, và Người dùng tại chỗ. Khách vắng lai có quyền xem các thông tin địa điểm và nội dung hỏi đáp công khai. Khi đã xác thực, người dùng có thể đóng góp đánh giá, đặt câu hỏi, thảo luận, lưu địa điểm và báo cáo vị trí. Người dùng tại chỗ là những cá nhân đã được hệ thống xác minh tọa độ vật lý trùng khớp với địa điểm, giúp nâng cao độ tin cậy của các câu trả lời do họ đóng góp. Hệ thống tương tác với Firebase để xác thực, SerpAPI để tìm kiếm, Goong để xử lý tọa độ, và Groq để sinh văn bản thông minh, trong khi toàn bộ dữ liệu nội tại được lưu giữ tại PostgreSQL.



Hình 11: Biểu đồ Ngữ cảnh Hệ thống (C4 Cấp 1)

7.4. C4 Level 2: Container Architecture

Hệ thống bao gồm ba container cốt lõi và các dịch vụ ngoại vi. Frontend được xây dựng bằng React và Vite, chạy trực tiếp trên trình duyệt của người dùng. Nhiệm vụ của nó là giao tiếp qua API REST định dạng JSON và đính kèm Firebase Auth Token vào các yêu cầu cần xác thực. Giao diện này kết nối trực tiếp đến Backend API, một ứng dụng NestJS tập trung toàn bộ logic nghiệp vụ, xác thực thẻ bảo mật, kết nối ngoại vi và thao tác dữ liệu. Dữ liệu được Backend API đọc ghi thông qua Database Container PostgreSQL, nơi lưu giữ chuẩn hóa toàn bộ nội dung do người dùng và trí tuệ nhân tạo tạo ra, đảm bảo tính bền vững vượt ra khỏi các kết quả tìm kiếm nhất thời.



Hình 12: Biểu đồ Kiến trúc Container (C4 Cấp 2)

7.5. Backend Modular Architecture

Máy chủ được thiết kế bám sát các business domains thay vì chia thư mục theo technical layers thuần túy. Cách tiếp cận nguyên khối phân hệ loại bỏ hoàn toàn sự phức tạp của việc trao đổi mạng nội bộ giữa các dịch vụ phân tán, nhưng vẫn duy trì tính tổ chức cực kỳ cao cấp cho mã nguồn.

Business Domain	Modules	Trách nhiệm Chính	Dữ liệu Trọng tâm
Khám phá	search, places	Điều phối truy vấn và đồng bộ hóa thông tin địa điểm.	Dữ liệu địa điểm
Định danh	users, Lớp xác thực Firebase	Quản lý hồ sơ cá nhân và kiểm chứng bảo mật.	Hồ sơ người dùng
Cộng đồng	questions, reviews, presence	Quản lý nội dung thảo luận, đánh giá và trạng thái vị trí.	Câu hỏi, Đánh giá, Trạng thái có mặt
Cá nhân hóa	saved-places, recommendations	Ghi nhớ sở thích và đề xuất địa điểm tương thích.	Danh sách yêu thích
Knowledge & AI	ai, rag	Phân loại ý định, sinh câu trả lời và tra cứu véc tơ.	Hội thoại, Vector Embeddings
Infrastructure	config, health, Truy xuất DB	Quản lý cấu hình môi trường, theo dõi sức khỏe hệ thống.	Environment Variables, Database Connection

Bảng 2: Tổ chức Business Domain tại Backend

7.6. Frontend Architecture

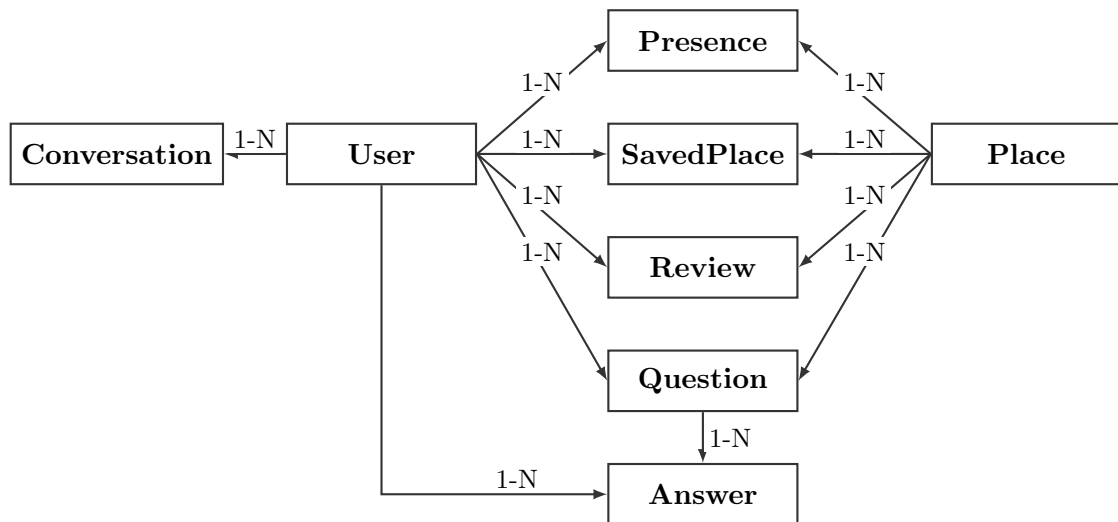
Frontend đóng vai trò là tầng điều phối quy trình làm việc và tương tác. Thay vì nhồi nhét logic nghiệp vụ, giao diện chỉ gọi các dịch vụ máy chủ và phản ánh trạng thái dữ liệu. React Router quản lý điều hướng và bảo vệ các protected routes. Context API đảm trách duy trì trạng thái toàn cục như thông tin xác thực, dữ liệu bản đồ và ngữ cảnh hội thoại.

Trang Chủ đảm nhận việc khám phá qua bản đồ, áp dụng bộ lọc và render các thẻ địa điểm cùng điểm đánh dấu. Trang Chi tiết Địa điểm đi sâu vào thông tin không gian, hiển

thị đánh giá, chuỗi hỏi đáp có gắn câu trả lời AI, nút xác nhận vị trí, nút lưu yêu thích và tích hợp luôn cửa sổ chat tư vấn chuyên biệt. Trang Đăng nhập là cửa ngõ duy nhất kết nối với hệ thống xác thực Firebase. Cuối cùng, Trang Hồ sơ cá nhân tóm tắt thông tin người dùng, trạng thái có mặt hiện tại và toàn bộ danh sách địa điểm đã lưu. Các thành phần giao diện như Bản đồ, Thẻ địa điểm, Khu vực Hỏi đáp được thiết kế độc lập, kết nối với nhau thông qua mạng lưới điều hướng và Error Boundary chặt chẽ.

7.7. Data Architecture

Cơ sở dữ liệu được chuẩn hóa nghiêm ngặt. Người dùng là trung tâm sở hữu vô số đánh giá, câu hỏi, câu trả lời, bản ghi vị trí, danh sách yêu thích và lịch sử hội thoại. Tương tự, Địa điểm là điểm neo cho các đánh giá, câu hỏi, vị trí và các tham chiếu hội thoại. Một câu hỏi chứa nhiều câu trả lời; các câu trả lời từ người dùng được liên kết nhân thân rõ ràng, trong khi các phản hồi được AI ghim lại tồn tại như những câu trả lời vô danh mang cờ hiệu hệ thống. Trạng thái vị trí lưu trữ chính xác thời điểm đến và đi. Bảng SavedPlace đóng vai trò cầu nối đa chiều giữa người dùng và địa điểm. Cấu trúc này cho phép các địa điểm ngoại vi từ API nhanh chóng hóa thân thành các thực thể bền vững trong hệ thống nội tại.



Hình 13: Sơ đồ Quan hệ Thực thể Đơn giản

7.8. Key Runtime Workflows

8.1 Luồng Tìm kiếm Người dùng thiết lập tham số tìm kiếm, giao diện truyền yêu cầu về máy chủ. Máy chủ kích hoạt Search Module, chuyển tiếp tham số cho SerpAPI (nhà cung cấp tìm kiếm chủ lực). Dữ liệu thô thu về được hệ thống loại bỏ rác, chuẩn hóa cấu trúc và hoàn trả về giao diện để vẽ bản đồ. Quyết định kiến trúc thiết lập SerpAPI làm nguồn tìm kiếm duy nhất, trong khi Goong hoàn toàn lui về chuyên trách giải mã tọa độ. Sự rạch ròi này ngăn chặn triệt để sự chồng chéo logic khi mở rộng hệ thống tìm kiếm trong tương lai.

8.2 Luồng Đồng bộ Khi truy cập trang chi tiết, giao diện kiểm tra xem mã định danh đang là chuỗi ngoài (ví dụ SerpAPI) hay UUID nội bộ. Nếu là chuỗi ngoài, máy chủ

tự động khởi tạo quá trình đồng bộ, chèn địa điểm vào cơ sở dữ liệu và biến nó thành một UUID tiêu chuẩn. Từ giây phút đó, mọi dữ liệu kéo theo như đánh giá, hỏi đáp, trạng thái vị trí đều được liên kết qua UUID này. Thiết kế này tạo ra một nhịp cầu liên mạch, biến một kết quả tìm kiếm bay hơi thành một thực thể neo đậu bền vững cho các hoạt động tương tác sâu của cộng đồng.

8.3 Luồng Xác thực Vị trí Người dùng nhấn nút xác nhận vị trí, trình duyệt lập tức trích xuất tọa độ GPS. Máy chủ đối chiếu khoảng cách toán học giữa tọa độ người dùng và tọa độ đích. Nếu hợp lệ, hệ thống ghi nhận trạng thái có mặt và cấp huy hiệu chứng nhận trên giao diện. Đây là một cơ chế thiết lập niềm tin cốt lõi, giúp nâng tầm giá trị của các câu trả lời do người tại chỗ cung cấp. Tuy nhiên, để bảo vệ quyền riêng tư, hệ thống luôn cho phép người dùng tự do rút lui bất kỳ lúc nào.

8.4 Luồng Hỏi đáp Tích hợp AI Ngay khi người dùng đăng câu hỏi, máy chủ lưu nháp và hiển thị ngay lập tức lên màn hình mà không cần chờ đợi. Ở chế độ nền, máy chủ âm thầm thu thập toàn bộ thuộc tính, tiện ích và các trích đoạn đánh giá của địa điểm để nạp vào cụm mô hình Groq. Mô hình phân tích và sinh ra câu trả lời tự nhiên, sau đó lưu cố định vào cơ sở dữ liệu. Lợi ích khổng lồ của thiết kế bất đồng bộ này là giao diện không bao giờ bị nghẽn mạng. Trí tuệ nhân tạo và con người cùng tồn tại song song trong một không gian thảo luận duy nhất.

8.5 Luồng Lưu Trữ Địa điểm Quá trình diễn ra ngắn gọn: máy chủ nhận yêu cầu và ghi chép lại mối liên kết vào bảng SavedPlace. Giao diện trang chỉ tiết và trang hồ sơ cập nhật tức thời. Quyết định này biến một ứng dụng tra cứu một lần thành một công cụ lập kế hoạch du lịch dài hạn mang tính cá nhân hóa cực cao.

7.9. Security and Access Control

Bảo mật hệ thống ủy thác cho Firebase ở client-side. Bất kỳ tác vụ cập nhật dữ liệu nào cũng phải đính kèm thẻ xác thực Firebase để NestJS xác thực qua Admin SDK. Các thao tác đọc thông tin công cộng được nới lỏng nhằm tối ưu trải nghiệm, nhưng mọi thao tác ghi, đặc biệt là quyền xóa đánh giá hay xóa câu hỏi, đều bị kiểm tra quyền sở hữu cực kỳ gắt gao. Khi một câu hỏi bị xóa, hệ thống kích hoạt cơ chế cascade delete dọn dẹp sạch sẽ các câu trả lời liên quan, đảm bảo không lưu lại rác dữ liệu. Trạng thái vị trí cũng có thể tự do xóa bỏ để bảo vệ quyền ẩn danh.

7.10. Runtime Configuration and Environment Strategy

Hệ thống áp dụng chiến lược Runtime Configuration thông qua Dependency Injection của NestJS để chuyển đổi mượt mà giữa môi trường Testing và Production. Việc branching logic được hạn chế tối đa; thay vào đó, toàn bộ cấu hình được trừu tượng hóa và nạp động từ các tệp `.env` tương ứng.

Đặc biệt ở chế độ Testing Mode, cơ chế phát triển được tối ưu hóa như sau:

- **Mocking External APIs:** Thay vì gửi request thực tế tốn chi phí và phụ thuộc mạng lưới, các dịch vụ ngoại vi (SerpAPI, Goong, Groq LLM) được hệ thống tự động inject các Mock Providers trả về kết quả định trước.
- **Isolated Test Database:** Chế độ kiểm thử sử dụng một schema cơ sở dữ liệu hoàn

toàn tách biệt. Dữ liệu được teardown (xóa sạch) sau mỗi chu kỳ chạy E2E Test, đảm bảo tính tất định tuyệt đối cho các kịch bản kiểm thử.

- **Ngắt lưu trữ rác:** Các chức năng ghi log hội thoại AI, xác thực Firebase thực tế hay đồng bộ dữ liệu tĩnh được vô hiệu hóa hoặc thay thế bằng cơ chế nội bộ để tăng tốc độ chạy test.

Cơ chế này cho phép các kỹ sư liên tục chạy unit test và e2e test ở môi trường cục bộ một cách độc lập, ổn định và không lo ngại giới hạn rate-limit của API bên ngoài.



Hình 14: Cơ chế Dependency Injection theo Môi trường Kiểm thử

7.11. Architectural Decisions and Trade-offs

Quyết định	Động lực	Lợi ích	Đánh đổi
Modular Monolith	Cần kiểm soát ranh giới rõ ràng nhưng thiếu nguồn lực vận hành phân tán.	Dễ dàng local deployment, dùng chung một kết nối cơ sở dữ liệu.	Không thể linh hoạt thay đổi quy mô từng phân hệ một cách độc lập.
Giao tiếp REST JSON	Tìm kiếm tiêu chuẩn công nghiệp an toàn và phổ biến nhất.	Hệ sinh thái công cụ dồi dào, gỡ lỗi mạng cực kỳ đơn giản.	Lãng phí một phần băng thông so với cấu trúc nhị phân nguyên thủy.
Xác thực Firebase	Tránh rủi ro tự quản lý mật khẩu mã hóa.	Chuẩn bảo mật cực cao, rút ngắn thời gian phát triển.	Bị phụ thuộc hoàn toàn vào dịch vụ đám mây của Google.
Đồng bộ Địa điểm Động	Cần lưu đánh giá cho địa điểm từ API ngoài.	Biến dữ liệu tạm thời thành dữ liệu bền vững nội bộ.	Bơm một lượng dữ liệu thụ động vào cơ sở dữ liệu.
Sinh câu trả lời AI bất đồng bộ	Tránh tình trạng chờ đợi quá lâu khi nạp ngữ cảnh vào LLM.	Giao diện phản hồi ngay lập tức cho người hỏi.	Phải bổ sung logic làm mới giao diện sau khi tạo xong.

Bảng 3: Các Quyết định Kiến trúc và Sự Đánh đổi

7.12. Current Limitations and Future Improvements

Kiến trúc bộc lộ một số giới hạn nhất định trong phạm vi đồ án hiện tại. Tính năng tìm kiếm bộ lọc tìm kiếm nâng cao theo phân loại địa điểm vẫn đang trong giai đoạn hoàn thiện. Luồng điều hướng và giao diện chỉ đường chưa đạt được sự mượt mà tối đa. Hệ thống cũng chịu sự ràng buộc bởi các dịch vụ cung cấp dữ liệu tìm kiếm, định vị và xử lý ngôn ngữ bên ngoài. Hơn nữa, chất lượng tư vấn của AI phụ thuộc tuyến tính vào lượng dữ liệu thực tế thu thập được; do đó, cơ chế kiểm duyệt và xếp hạng bình chọn cho các câu hỏi sẽ cần được bổ sung để nâng cấp chất lượng tri thức cộng đồng.

7.13. Summary

Kiến trúc hệ thống thể hiện sự rạch ròi tuyệt đối giữa việc render giao diện, xử lý luồng nghiệp vụ cốt lõi, lưu trữ dữ liệu bền vững và giao tiếp ngoại vi. Việc ứng dụng linh hoạt mô hình Modular Monolith giúp nhóm phát triển đạt được sự cân bằng hoàn hảo giữa tính dễ bảo trì và sự tinh giản trong vận hành. Nhờ những cơ chế đột phá như xác thực tọa độ vị trí vật lý và quy trình đồng bộ dữ liệu ngoại vi theo thời gian thực, nền tảng không chỉ là một công cụ tìm kiếm đơn thuần mà đã thực sự chuyển mình thành một mạng lưới tri thức cộng đồng an toàn, đáng tin cậy và mang đậm tính cá nhân hóa sâu sắc.

8. Implementation

8.1. From Design Model to Module Decomposition

Từ các mô hình đã xây dựng ở phần Thiết kế Hệ thống (C4 Model, ERD, sơ đồ tuần tự), nhóm tiến hành phân rã thành các module triển khai theo ba bước:

1. **Phân rã chức năng:** Các khối chức năng trong C4 Container → 10 module NestJS, mỗi module gồm tầng giao tiếp (API), tầng logic (xử lý nghiệp vụ) và tầng dữ liệu (DTO).
2. **Ánh xạ dữ liệu:** Mô hình ERD → 13 lược đồ Prisma với quan hệ, ràng buộc và chỉ mục. Các thao tác vector nhúng (pgvector) được xử lý qua SQL thô do hạn chế của ORM.
3. **Ánh xạ quy trình:** Sơ đồ tuần tự → workflow registry dạng JSON và các lớp điều phối AI. Mỗi ca sử dụng tương ứng với một workflow cụ thể trong registry.

Các module được tổ chức thành bốn nhóm:

Nhóm	Module	Nguồn gốc
Dịch vụ lõi	Users, Places, Reviews, Search, Health	Kế thừa từ core-service (NestJS)
AI và Tìm kiếm	AI, Recommendations	Chuyển đổi từ FastAPI sang NestJS
Mở rộng MVP	RAG, Presence, Contributions	Phát triển mới
Dùng chung	PrismaService, ToolRegistry, Global Filters	Toàn cục, dùng chung qua DI

8.2. Architecture Migration: Microservices → Modular Monolith

Hệ thống nguyên bản vận hành với bốn dịch vụ riêng rẽ (Nginx Gateway, Core-service, AI-service, Recommendation-service). Kiến trúc này bộc lộ các vấn đề:

- **Chi phí mạng:** Mỗi yêu cầu chat trải qua ba chặng HTTP, gây độ trễ tích lũy.
- **Trùng lặp logic:** Xác thực, xử lý lỗi, cấu hình bị sao chép giữa các dịch vụ.
- **Phân mảnh kiểu:** Python ↔ TypeScript yêu cầu tuần tự hóa thủ công, dễ sai sót.
- **Phức tạp vận hành:** Bốn Docker image, bốn pipeline triển khai riêng biệt.

Giải pháp: Hợp nhất thành một monolith dạng module NestJS duy nhất. Các lời gọi HTTP liên dịch vụ được thay thế bằng phương thức gọi trực tiếp qua dependency injection, loại bỏ Gateway Nginx trung gian.

8.3. Challenges and Solutions

8.3.1 LLM Non-determinism

Thách thức: LLM sinh phản hồi không nhất quán về định dạng, dễ hallucination dữ liệu, và độ trễ khó dự đoán.

Giải pháp — Kiến trúc Bốn Lớp:

- **Lớp 1 (Router):** LLM với JSON Mode → phân loại ý định, trích xuất tham số có cấu trúc. Thất bại → mặc định GENERAL_CHAT.
- **Lớp 2 (Workflow Engine):** Thực thi DAG các bước tất định, không gọi LLM.
- **Lớp 3 (Tool Layer):** Mỗi công cụ có hợp đồng đầu ra chuẩn hóa, không ném ngoại lệ.
- **Lớp 4 (Composer):** Neo phản hồi Markdown vào dữ liệu thực tế từ công cụ, giảm hallucination.

Lợi ích: Workflow mới có thể thêm dưới dạng JSON, không cần biên dịch lại.

8.3.2 SSE Streaming via POST

Thách thức: SSE chuẩn chỉ hỗ trợ GET, trong khi endpoint chat cần gửi JSON payload và token xác thực.

Giải pháp — Mô hình hai pha:

- Sử dụng `fetch()` với `ReadableStream` thay vì `EventSource`, cho phép POST và đính kèm token.
- Pha 1: Gửi dữ liệu cấu trúc (`searchAction`) ngay sau khi công cụ thực thi — cho phép frontend render bản đồ trước.
- Pha 2: Truyền token LLM dưới dạng delta SSE.
- Header `X-Accel-Buffering`: no để vô hiệu hóa bộ đệm Nginx khi triển khai sau reverse proxy.

8.3.3 Multi-Map Provider Integration

Thách thức: Mỗi API bản đồ trả về định dạng khác nhau; lỗi một nhà cung cấp không được ảnh hưởng toàn bộ tìm kiếm.

Giải pháp:

- Chuẩn hóa giao diện đầu ra cho mọi nhà cung cấp.
- Gọi song song qua `Promise.all` với `.catch()` riêng biệt — đảm bảo cách ly lỗi.
- Chuỗi dự phòng: Goong → OSM → mảng rỗng.

8.3.4 Vector Embedding and Prisma

Thách thức: Prisma chưa hỗ trợ nguyên sinh kiểu `vector` của PostgreSQL.

Giải pháp: Khai báo cột `embedding` dưới dạng `Unsupported("vector")`, thao tác qua SQL thô (`$queryRawUnsafe`).

8.3.5 Firebase Authentication

Thách thức: Tích hợp Firebase JWT vào vòng đời NestJS guard mà không sao chép logic xác thực trên mọi controller.

Giải pháp: Triển khai guard với `CanActivate`: trích xuất `Authorization` header → xác thực token qua Firebase Admin SDK → gắn decoded token vào request. Frontend Axios interceptor tự động đính kèm token.

8.4. Other Challenges

Xử lý đồng thời: Ba mẫu thiết kế được áp dụng: (1) `Promise.all` với catch riêng biệt; (2) ghi vào khóa khác nhau trong shared state để tránh race condition; (3) Active Flag pattern để tránh cập nhật state sau khi component unmount.

Triển khai Docker: Ba container (frontend/Nginx, backend/Node 22, PostgreSQL/pgvector) với chiến lược build đa tầng tận dụng layer cache. Docker override cho môi trường phát triển sử dụng volume ẩn danh tránh xung đột `node_modules`.

Quản lý bản đồ frontend: Chuyển đổi layer bằng opacity, MarkerClusterGroup với chunkedLoading cho hiệu năng, CustomEvent thay vì prop drilling cho tích hợp AI-Search.

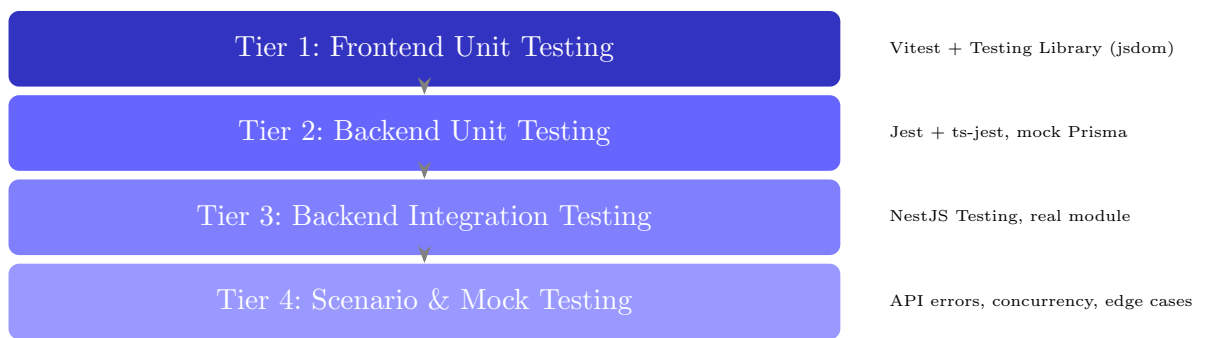
8.5. Conclusion

Quá trình triển khai cho thấy việc chuyển đổi từ mô hình thiết kế sang mã nguồn đòi hỏi liên tục giải quyết các vấn đề thực tế: tính phi tất định của LLM, hạn chế của ORM với kiểu dữ liệu đặc thù, và đảm bảo tính nhất quán xuyên suốt các tầng. Mười hai vấn đề đã được ghi nhận với các mức độ ưu tiên khác nhau.

9. Testing

9.1. Testing Strategy

Nhóm áp dụng chiến lược kiểm thử đa tầng, bao phủ từ đơn vị đến tích hợp, với mục tiêu đảm bảo chất lượng cho toàn bộ hệ thống:



Nguyên tắc: Mock toàn bộ tầng dữ liệu và dịch vụ ngoài ở unit test; kiểm thử tích hợp sử dụng module thật với dependency injection đầy đủ. Mỗi tầng kiểm thử bổ sung và mở rộng cho tầng trước.

9.2. Testing Scope

9.2.1 Frontend — UI and Interaction

- **Tầng dịch vụ:** Kiểm thử 12 dịch vụ frontend bao gồm API chat, tìm kiếm, địa điểm, đánh giá, hiện diện, đóng góp tệp, gợi ý, RAG, người dùng — xác nhận mỗi service gọi đúng endpoint, xử lý đúng dữ liệu trả về và bắt lỗi HTTP.
- **Context:** Kiểm thử 3 context (Auth, TravelData, Theme) — xác nhận provider render đúng, cung cấp giá trị mặc định và cập nhật state khi có thay đổi.

- **Hooks:** Kiểm thử hook streaming AI — ghép chunk delta SSE, xây dựng prompt ngữ cảnh, xử lý lỗi luồng. Kiểm thử ánh xạ lỗi Firebase — dịch 4 mã lỗi (permission-denied, unauthenticated, network-request-failed, unknown) sang thông báo tiếng Việt.
- **Component:** Kiểm thử 25 component giao diện chính: MapComponent (hiển thị marker, cluster), ChatPanel (gửi/nhận tin nhắn, streaming), PlaceCard (hiển thị thông tin, nút tương tác), SearchBar (tìm kiếm, gợi ý), các component wizard-inputs (LocationPicker, BudgetSelector, TypeSelector) và chat subcomponents (MessageList, StreamingText).
- **Trang:** Kiểm thử 5 trang: HomePage, SearchPage, PlaceDetailPage, ProfilePage, AdminPage — xác nhận layout, điều hướng và tích hợp component.

9.2.2 Backend — Business Logic and Integration

- **Dịch vụ (Services):** Kiểm thử 17 service backend bao phủ toàn bộ module: Người dùng (CRUD, upsert, tìm kiếm), Địa điểm (CRUD, lọc, fuzzy match), Đánh giá (CRUD, điểm số), Tìm kiếm (đa nhà cung cấp, hợp nhất, khử trùng lặp), Gợi ý (tính điểm, xếp hạng, trọng số), Hiện diện (join/leave/get, tải đồng thời), Đóng góp (tạo/lấy/cập nhật tệp), RAG (lưu/truy xuất chunk, xây dựng ngữ cảnh), Hội thoại AI (danh sách/tạo/xóa/tin nhắn), NLP (trích xuất bộ lọc từ văn bản tiếng Việt).
- **Controller:** Kiểm thử 12 controller — xác nhận route đăng ký đúng, validation pipe hoạt động, response format chuẩn.
- **Công cụ AI:** Kiểm thử 10 tool — SearchPlaces (tìm kiếm lại), RecommendPlaces (xếp hạng), GeocodeAnchor (mã hóa địa lý), ProximityChecker (khoảng cách), NearbyAmenities (tiện ích lân cận), và các tool bổ trợ. Mỗi tool được kiểm tra cả luồng thành công và thất bại.
- **Guard/Filter/Interceptor:** Kiểm thử FirebaseAuthGuard (thiếu header, sai định dạng, token không hợp lệ), AllExceptionsFilter (HttpException, lỗi không xác định), LoggingInterceptor (định nghĩa, phương thức).

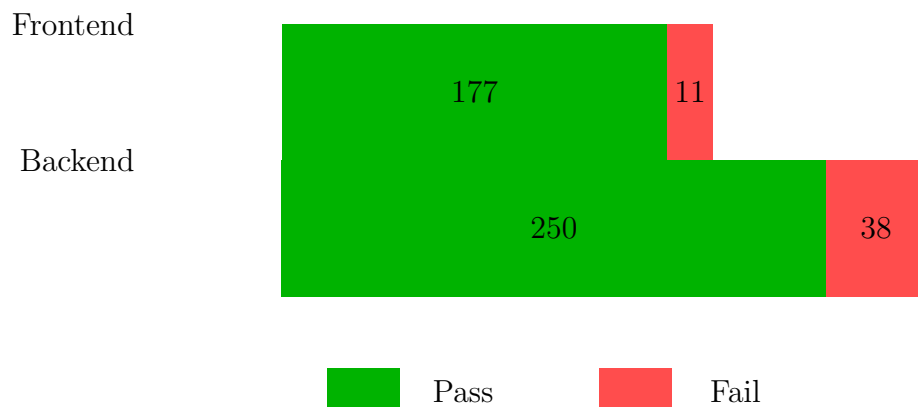
9.2.3 Specific Scenarios and Mocking

- **Giả lập lỗi nhà cung cấp ngoài:** Khi Goong API thất bại, hệ thống tự động chuyển sang OSM; khi cả hai thất bại, trả về kết quả rỗng an toàn. Một nhà cung cấp lỗi không ảnh hưởng đến kết quả từ các nhà cung cấp khác.
- **Xử lý workflow thất bại:** Router phân tích lỗi → dự phòng GENERAL_CHAT; tool lỗi → đánh dấu lỗi và tiếp tục thực thi workflow.
- **SSE streaming đầu cuối:** Xác minh kết nối, nhận chunk delta, xử lý finish, buffer ghép mảnh.
- **Dữ liệu rỗng và trường hợp biên:** Tìm kiếm không kết quả, hội thoại mới, người dùng chưa có profile, địa điểm không có đánh giá.

- **Tải đồng thời:** 100 người dùng join cùng lúc vào cùng địa điểm — kiểm tra tính toàn vẹn của cấu trúc dữ liệu.

9.3. Testing Results

Kết quả kiểm thử tổng thể được thống kê như sau:



Frontend — 177/188 đạt (94,1%)

12 services, 3 contexts, 3 hooks, 5 pages, 25 components

Backend — 250/288 đạt (86,8%)

17 services, 12 controllers, 10 tools, 3 guards/filters/interceptors

Tổng thể — 427/476 đạt (89,7%)

Phân bố kiểm thử theo nhóm chức năng backend:



Phân bố kiểm thử theo nhóm chức năng frontend:

Nhóm chức năng	Số dịch vụ/Component	Kết
Dịch vụ API (aiService, searchService, placeService, ...)	12 services	11/11
Ngữ cảnh (AuthContext, TravelDataContext, ThemeContext)	3 contexts	2/3
Hooks (streamingChat, firestoreError)	3 hooks	3/3
Component giao diện (Map, Chat, PlaceCard, SearchBar, ...)	25 components	25/25
Trang (Home, Search, PlaceDetail, Profile, Admin)	5 pages	5/5

Ghi chú: 11 ca thất bại frontend và 38 ca thất bại backend đều liên quan đến mock Firebase/Firestore chưa hoàn chỉnh — không phải lỗi logic nghiệp vụ. Tất cả module chức năng chính đều được kiểm thử thành công.

9.4. Post-Testing Improvements

Quá trình kiểm thử đã phát hiện và dẫn đến các cải tiến sau:

- **Cải tiến xử lý lỗi:** Chuẩn hóa phản hồi lỗi thành cấu trúc JSON thống nhất với timestamp, loại bỏ try/catch trùng lặp ở controller.
- **Cải tiến khử trùng lặp:** Bổ sung khóa mờ theo tọa độ (lat:lng độ chính xác 3 chữ số thập phân) bên cạnh khóa chính xác, giúp phát hiện trùng lặp khi ID nhà cung cấp khác nhau.
- **Cải tiến buffer SSE:** Cả backend và frontend được đồng bộ buffer pattern để xử lý chunk SSE bị phân mảnh do TCP.
- **Cải tiến Presence:** Thêm kiểm tra tải 100 người dùng đồng thời, đảm bảo cấu trúc dữ liệu hoạt động ổn định dưới tải cao.
- **Cải tiến NLP:** Tham số hóa 14 văn bản tiếng Việt thực tế, bao phủ đa dạng biến thể ngôn ngữ (tên tỉnh thành, loại hình, mức giá).

9.5. Future Development

Các khoảng trống kiểm thử đã được xác định:

- Kiểm thử tích hợp với PostgreSQL + pgvector thực (hiện đang mock Prisma).
- Pipeline CI/CD tự động hóa quy trình kiểm thử.
- Kiểm thử hiệu năng với tập dữ liệu lớn (10^5 + địa điểm).
- Kiểm thử người dùng có tổ chức.

10. Demo

11. Deployment

12. Logbook: Plan and Actual Workload

12.1. Plan Workload

The initial planned workload for the project was structured across six weeks, focusing on a clear progression from system design to final deployment. The plan was distributed as follows:

- **Week 1 – 2:** Requirement gathering, problem decomposition, and system architecture design. Prototyping UI mockups.
- **Week 3:** Frontend foundation setup, integrating Mapbox/Goong maps, and implementing Firebase Authentication.
- **Week 4:** Backend API construction using NestJS, database modeling with Prisma and PostgreSQL, and integration with external search APIs (SerpAPI).
- **Week 5:** Implementation of AI features, including the Vector database (pgvector) setup, RAG pipeline, and AI-pinned Q&A workflows.
- **Week 6:** End-to-end testing, UI/UX refinement, bug fixing, and finalizing the project report.

12.2. Actual Workload

During the implementation phase, the team encountered various technical challenges that altered the initial timeline, resulting in the actual workload diverging from the plan:

- **Week 1 – 2:** Architecture design took slightly longer than anticipated due to the strategic pivot from a Microservices architecture to a Modular Monolith, which required redesigning the component interactions.
- **Week 3:** Frontend setup and map integration proceeded smoothly, but coordinating Firebase Authentication with protected routes required extra debugging.
- **Week 4:** Database modeling was completed successfully. Integrating SerpAPI was straightforward, but data normalization for hybrid search demanded additional logic.
- **Week 5:** A significant amount of unplanned time was invested in fine-tuning the RAG pipeline. Ensuring that the Groq LLM did not hallucinate place details required extensive prompt engineering and context chunking adjustments.

- **Week 6:** Extensive testing was conducted, specifically mocking location data for the physical check-in feature. Report writing in LaTeX also consumed a considerable portion of the final week.

12.3. Workload Distribution

100% of the total workload is divided equally among the group members. The team worked collaboratively, with pair programming and regular syncs ensuring that no single component became a bottleneck. The table below illustrates the percentage of the total workload each group member has done, along with their estimated working hours.

No.	Student ID	Name	Percentage of Workload	Working Hours
1	24120033	Đào Tiến Đạt	20%	45 hours
2	24120040	Dương Hoài Đức	20%	45 hours
3	24120304	Trần Thị Bích Hạnh	20%	45 hours
4	24120310	Nguyễn Đình Hiếu	20%	45 hours
5	24120483	Hoàng Văn Trường	20%	45 hours

Bảng 4: Group Members Workload Distribution

13. AI usage and declaration

14. Conclusion and Future Work

14.1. Conclusion

Trong quá trình thực hiện đề tài, nhóm đã xây dựng thành công ứng dụng web toàn diện với kiến trúc hiện đại, tích hợp các công nghệ AI tiên tiến. Hệ thống được phát triển trên nền tảng React và NestJS, kết hợp với Prisma ORM, Firebase Authentication, và các dịch vụ AI đa nền tảng (Gemini, Cloudflare AI, FreeModel).

Các chức năng đã được triển khai thành công bao gồm:

- Chức năng tìm kiếm qua chatbot
- Chức năng so sánh các địa điểm lưu trú

- Chức năng trích xuất thông tin chi tiết và đánh giá của AI về một địa điểm lưu trú cụ thể

14.2. Acquired Knowledge and Skills

Môn học và đồ án lần này đã mang lại cho chúng em những bài học vô cùng thực tế, giúp thu hẹp khoảng cách giữa lý thuyết và ứng dụng:

- **Vận dụng 4 trụ cột cốt lõi của Tư duy Máy tính:** Thông qua đồ án, chúng em đã làm chủ và áp dụng thành công mô hình tư duy này để giải quyết bài toán thực tế. Cụ thể, chúng em thực hiện
 1. *Phân rã bài toán* (Decomposition) để chia nhỏ hệ thống thành các module độc lập (*AiModule* và *RecommendationsModule*);
 2. *Nhận biết mẫu* (Pattern Recognition) để tìm ra khuôn mẫu trong hành vi tìm kiếm của khách hàng;
 3. *Trừu tượng hóa* (Abstraction) để lọc bỏ thông tin nhiễu, trích xuất tham số cốt lõi qua *Intent Parsing*;
 4. *Thiết kế thuật toán* (Algorithm Design) để xây dựng luồng vận hành tối ưu cho Chatbot.

Các kỹ năng này giúp chúng em định hình được tư duy logic vững chắc cho các dự án phức tạp sau này.

- **Tư duy khai thác Trí tuệ nhân tạo:** Chúng em đã học được cách viết kỹ năng Prompt Engineering và cách ứng dụng AI một cách hiệu quả, biến AI thành một công cụ đồng hành đắc lực để tối ưu hóa việc tìm kiếm thông tin thay vì lạm dụng một cách máy móc.
- **Làm quen với các công nghệ mới:** Đồ án là cơ hội để chúng em tiếp cận và biết cách sử dụng hàng loạt công cụ, ngôn ngữ và nền tảng cốt lõi trong phát triển phần mềm hiện đại như HTML, JavaScript, nền tảng cơ sở dữ liệu Firebase, bản đồ OpenStreetMap,... Dù thời gian có hạn và chưa thể tìm hiểu quá sâu sắc từng công nghệ, nhưng đây là nền móng cực kỳ quan trọng cho các môn học chuyên ngành tiếp theo.
- **Kỹ năng làm việc nhóm và Quản lý mã nguồn:** Biết cách sử dụng GitHub để quản lý các phiên bản code, phối hợp phân chia công việc trong nhóm một cách hệ thống.
- **Tư duy Hệ thống:** Hiểu rõ bức tranh toàn cảnh về cách một hệ thống phần mềm vận hành trong thực tế (cách Frontend tương tác với Backend, cách gọi API và truy vấn dữ liệu từ Database).

14.3. Future Development Directions

Nhằm nâng cấp và hoàn thiện dự án thành một sản phẩm có tính ứng dụng thực tế cao hơn, các mục tiêu trong tương lai bao gồm:

- **Nghiên cứu sâu về RAG và LLM:** Tối ưu hóa sâu hơn kỹ thuật Retrieval-Augmented Generation (RAG) kết hợp với các mô hình ngôn ngữ lớn (như Groq) để nâng cao độ chính xác, giảm thiểu hiện tượng "ảo tưởng" (hallucination) của Chatbot khi trả lời khách hàng.
- **Tối ưu hóa UI/UX:** Thiết kế lại giao diện người dùng thân thiện, trực quan hơn trên cả hai nền tảng Web và Mobile, tối ưu hóa các bộ lọc nâng cao để nâng cao trải nghiệm của các nhóm đối tượng Target Users.